

NJALA UNIVERSITY



Master of Science in Computer Science

CSP610 — Object-Oriented Programming Techniques

Smart Library Management System

Assignment Report

Module: CSP610 Object-Oriented Programming Techniques

Language: Java 17 (preview: pattern-matching switch)

Build Tool: Maven 3.x

Testing: JUnit 5.10, Mockito 5.5

Coverage: JaCoCo 0.8.11

Tutor: Dr. Abdul Joseph Fofanah

Submission Date: 30th May 2026

By

92604

Joyce Thomas

92607

Milton Jusu

Table of Contents

Table of Contents	2
1. Abstract	4
2. Introduction and Design Overview	5
2.1 System Context	5
2.2 Package Architecture	6
2.3 Design Principles	6
3.1 Original Violations.....	7
3.2 Refactored Abstractions.....	8
3.3 Liskov Substitution	9
3.4 Interface Segregation	9
3.5 Business Rules	10
4. Design Pattern Justifications with UML Diagrams (Task 2).....	11
4.1 Builder: LoanBuilder and ReservationBuilder	11
4.2 Decorator: Notification Stack	12
4.3 Observer: LibraryEventBus	14
4.4 Dual Notification and Event Models	15
5. Generics and Type System Discussion (Task 3).....	17
5.2 PECS	17
5.3 Type Erasure and Mitigation	17
5.4 Repository<T, ID>	19
6.1 LibraryAnalytics: Seven Required Methods.....	19
6.2 Custom Collector	20
6.3 Method References and Composition	20
7.1 ReturnProcessor: CompletableFuture Pipeline	20
7.2 ConcurrentInventory	21
7.3 Race, Deadlock, and Livelock	21
8.2 Record vs Mutable Class	23
8.3 Pattern Matching and Text Blocks.....	24
8.4 Java Platform Module System	24
9.1 Test Suite	24
9.2 JaCoCo Coverage Metrics	26
10. Conclusion and Lessons Learned.....	28

Key Lessons	29
Limitations	29
Future Work	29
Peer-Reviewed Articles	30
Textbooks and Standards	31
Appendix A: Figures and UML Sources	32
Appendix B: Regenerating Artefacts	32
Appendix C: Related Documents.....	33

1. Abstract

This report covers the design and implementation of SmartLib, a modular university library management system built in Java 17 for the CSP610 Object-Oriented Programming Techniques assignment. We began with a provided `LibraryManager` class that crammed fine calculation, email delivery, file persistence, and report generation into one monolithic unit. The core of the refactoring was to break this apart into well-defined interfaces (`FineCalculator`, `NotificationService`, `LoanRepository`, and `ReportGenerator`), coordinated through `LibraryService` and `LoanManagementService` by constructor injection (Parnas, 1972; Martin, 2017).

The business rules, namely borrowing limits, loan durations, a \$0.50 per-day fine rate, a three-day reservation expiry, and a \$10 outstanding-fine threshold, live inside `BorrowingPolicy` implementations and are checked by `DomainBusinessRulesTest`. For Task 2 we applied one Gang-of-Four pattern to each kind of problem it suits: Builder for object construction, Decorator for cross-cutting notification concerns, and Observer for event-driven reactions. Tasks 4 and 5 cover stream-based analytics with a custom Collector, asynchronous batch processing with `CompletableFuture`, and thread-safe inventory management built on a `ReentrantReadWriteLock` (Naftalin & Wadler, 2006; Goetz et al., 2006).

Task 6 uses a sealed `LoanResult` type with exhaustive pattern-matching switch expressions, records for immutable value objects, and a Java Platform Module System declaration. In total the suite runs 62 tests across seventeen JUnit classes with no failures, reaching 67.6% instruction coverage under JaCoCo. Two parts are deliberately unfinished. CSV file I/O and PDF report generation are left as stubs, and rather than hide that, we set it out plainly in Section 10.

2. Introduction and Design Overview

SmartLib manages the core operations of a university library: cataloguing books, registering members, processing loans, handling reservations, calculating fines, and delivering notifications. Instead of organising the code around technical layers like persistence or presentation, we structured the packages to mirror the six assignment tasks directly. The payoff is traceability: each design decision maps back to a specific requirement, and each concern can be tested on its own.

Package	Assignment Task	Primary Types
smartlib.domain	Task 1: SOLID core	LibraryService, entities, policies, repositories
smartlib.patterns	Task 2: GoF patterns	LoanBuilder, decorators, LibraryEventBus
smartlib.generics	Task 3: Generics	Catalogue<T>, TokenType, PECS utilities
smartlib.functional	Task 4: Streams	LibraryAnalytics, LoanSummaryCollector
smartlib.concurrent	Task 5: Concurrency	ReturnProcessor, ConcurrentInventory
smartlib.modern	Task 6: Modern Java	LoanResult, LoanResultHandler, ModernLibraryFormatter

Table 1: Package-to-Task Mapping

2.1 System Context

At its simplest, the system lets a member borrow or return a book. Every borrowing action runs through a BorrowingPolicy, which decides how many items the member may hold and for how long. Return a book late and the system calculates a fine and notifies the member; ask for a book that is out and it creates a reservation automatically. The infrastructure, comprising in-memory data stores and a console-based email simulation, sits behind interfaces, so the domain logic can be exercised in tests without touching any external system.

2.2 Package Architecture

Figure 1 shows how the domain package acts as the orchestration hub, with `LibraryService` and `LoanManagementService` depending on abstractions that the other packages implement or extend. The patterns package connects through two adapter classes: `DomainNotificationAdapter` bridges the notification models used in Tasks 1 and 2, while `LibraryEventBusPublisher` translates domain return events into the event bus messages consumed by the observer listeners. This bridging is discussed further in Section 4.4.

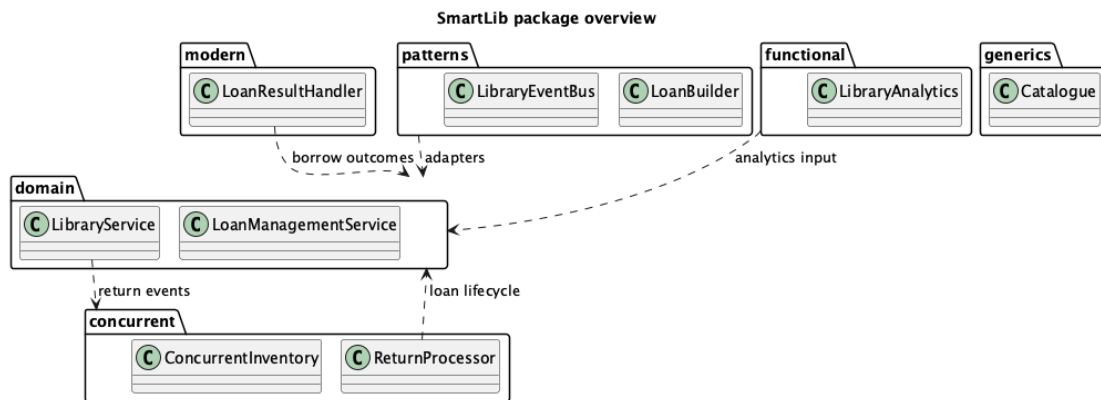


Figure 1: SmartLib Package Architecture

2.3 Design Principles

A handful of SOLID principles shaped the architecture. Dependency Inversion runs through almost everything: `LibraryService` takes its dependencies by constructor injection and never names a concrete transport class such as an SMTP client or a `FileWriter` (Martin, 2017). Open/Closed holds because a new report format or notification behaviour means writing a new class, not editing an old one (Gamma et al., 1994). Single Responsibility is enforced at the package level, with fine calculation, notification, and persistence each owning a distinct type. The domain model got firmer too: swapping the untyped `List<Object[]>` containers for real `Book`, `Member`, and `Loan` entities wiped out a whole category of type-safety bugs. One more thing worth flagging is that two return workflows live side by side: a synchronous path through `LibraryService.returnBook` for the simple cases and an asynchronous batch pipeline through `ReturnProcessor` for concurrent workloads, both covered in Section 7.

3. SOLID Analysis and Refactoring (Task 1)

3.1 Original Violations

Held up against the SOLID criteria, the original LibraryManager shows a cluster of related problems, and nearly all of them come back to one cause: the class simply does too much. The returnBook method is the clearest breach of Single Responsibility: in a single method body it works out an overdue fine, sends an email, and writes a record to a CSV file. Touch the fine logic and you risk breaking the email path; change the persistence format and you are editing notification code in the same breath. generateReport repeats the pattern, emitting PDF, CSV, and console output all from one place.

The Open/Closed Principle is violated as a direct consequence: adding a new report format means editing generateReport and re-testing all existing formats alongside it, rather than simply supplying a new implementation.

Dependency Inversion is absent throughout. Email is dispatched by directly constructing JavaMail/SMTP objects inside the method, and loan records are written by constructing a FileWriter explicitly. Neither dependency is injectable or replaceable without modifying the class. The Interface Segregation Principle is implicitly broken because there are no interfaces at all; any code that needs the library must depend on the entire LibraryManager. Finally, the overdue date calculation logic appears in both returnBook and calculateFine, creating a duplication that means a business rule change requires two coordinated edits rather than one.

Principle	Location (spec)	Violation
S	returnBook	Fine calculation, email notification, and CSV persistence combined in one method
S	generateReport	PDF, CSV, and console output produced from a single method
O	generateReport	Adding a new report format requires editing the existing method
D	Email block	JavaMail/SMTP constructed directly inside the method
D	Persistence	FileWriter to loans.csv constructed directly inside the method

I	(implicit)	No interfaces defined; all clients coupled to a single concrete class
DRY	returnBook / calculateFine	Overdue date logic duplicated across two methods

Table 2: SOLID Violations in the Original LibraryManager

3.2 Refactored Abstractions

The following abstractions address each of the violations identified above. Each interface has a single, clearly-bounded responsibility, and all dependencies are supplied through constructor injection.

Abstraction	Implementation(s)	SOLID Benefit
FineCalculator	StandardFineCalculator	S/O: overdue arithmetic in one place
NotificationService	EmailNotificationService (domain)	D: transport details absent from LibraryService
LoanRepository	InMemoryLoanRepository, CsvLoanRepository	D: persistence is a swappable interface
ReportGenerator	Pdf, Csv, Console	O: new format added by registration, not branching
LibraryService	Constructor-injected coordinator	S/D: synchronous return workflow and reporting
LoanManagementService	Loan lifecycle management	S: used exclusively by ReturnProcessor

Table 3: Refactored Abstractions and Their SOLID Benefits

One limitation worth noting is that CsvLoanRepository does not yet perform real file I/O; it delegates entirely to InMemoryLoanRepository. The type exists and satisfies the interface contract, so

LibraryService is unaffected, and replacing the delegate with a genuine file-backed implementation in future would require no changes to the service layer.

The refactored returnBook method below illustrates this delegation clearly. Each collaborator is called in sequence: the fine is calculated, the loan is marked as returned and saved, the member is notified, and an event is published. The method has no knowledge of email transport, file formats, or event bus internals.

```
public double returnBook(String loanId) {
    validateLoanId(loanId);
    Loan loan = loanRepository.findById(loanId)
        .orElseThrow(() -> new LoanNotFoundException(loanId));
    double fine = fineCalculator.calculateFine(loan);
    Loan returnedLoan = loan.markReturned();
    loanRepository.save(returnedLoan);
    notificationService.sendReturnConfirmation(
        returnedLoan.memberEmail(), returnedLoan.id(), fine);
    eventPublisher.publishBookReturned(
        returnedLoan.id(), returnedLoan.memberEmail(), fine);
    if (fine > 0.0) {
        eventPublisher.publishFineImposed(
            returnedLoan.id(), returnedLoan.memberEmail(), fine);
    }
    return fine;
}
```

Code 1: Refactored returnBook Method in LibraryService

3.3 Liskov Substitution

The three BorrowingPolicy implementations, StandardPolicy, PremiumPolicy, and StudentPolicy, are fully substitutable. Each honours the same contract by implementing maxBorrowLimit, loanDurationDays, dailyFineRate, and canBorrow without altering any externally observable invariant. Neither LibraryService nor LoanBuilder references a concrete policy class, relying instead on the interface, which is consistent with the behavioural subtyping notion described by Liskov and Wing (1994).

3.4 Interface Segregation

The interfaces in SmartLib are deliberately narrow. ReportGenerator exposes only format() and generate(List<Loan>), so a report client does not need to know anything about notifications or persistence. Keeping FineCalculator separate from LoanRepository means that a class needing only to calculate fines is not forced to implement unrelated repository methods, which was precisely the kind of fat interface the original LibraryManager represented.

3.5 Business Rules

LibraryServiceTest uses Mockito to verify that returnBook correctly delegates to each collaborator, confirming that the SOLID decomposition holds at the boundary between the service and its dependencies without requiring any concrete infrastructure to be instantiated.

Business Rule	Assertion
Borrow limits	Standard 3, Premium 10, Student 5; at the limit, canBorrow returns false
Loan durations	14 days (Standard), 28 days (Premium), 21 days (Student)
Fine rate	\$0.50 per day; 7 overdue days yields a \$3.50 fine
Reservation expiry	Reservation expires after 3 days if unclaimed
Fine threshold	Outstanding fine exceeding \$10 blocks further borrowing

Table 4: Business Rules Verified by DomainBusinessRulesTest

4. Design Pattern Justifications with UML Diagrams (Task 2)

4.1 Builder: LoanBuilder and ReservationBuilder

The case for Builder here is simple. A Loan has two mandatory fields, the borrowing member and the target book, but a shifting set of optional ones: notes, renewal count, a reference code. Build a Loan without a builder and you are stuck choosing between telescoping constructors or parameter lists peppered with nulls, and in either case a deliberately empty field looks exactly like one you forgot to fill in.

The fluent builder solves this by deferring validation to the build() call, at which point it assigns a UUID loan identifier, records the current date as loanDate using `LocalDate.now()`, and derives the dueDate from the borrowing policy duration. This means the caller cannot accidentally create a Loan without a due date, and the logic for computing that date lives in exactly one place. ReservationBuilder follows the same pattern for reservations. Tests in `LoanBuilderTest` and `ReservationBuilderTest` cover the minimal-field case, the fully-populated case, and the guard that a negative renewalCount throws an `IllegalArgumentException`.

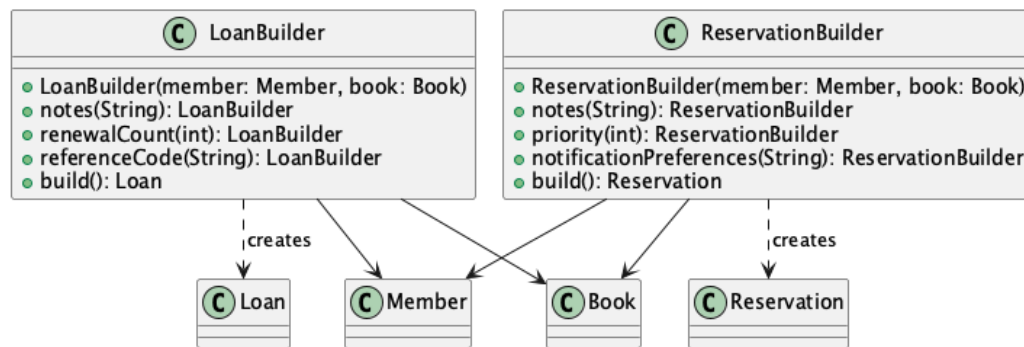


Figure 2: Builder Pattern Class Diagram

Q2.1b: Why Builder over Telescoping Constructors or JavaBeans?

Telescoping constructors require one overload per optional combination: `Loan(member, book)`, `Loan(member, book, notes)`, `Loan(member, book, notes, renewalCount)`, and so on. With three optional fields there are already eight overloads; adding a fourth doubles the count. Call-sites become unreadable, since `new Loan(member, book, null, 2, null)` gives no indication of which argument is which.

The JavaBeans pattern (no-arg constructor + setters) solves readability but introduces a different problem: the object exists in a partially constructed, inconsistent state between construction and the last set call. A thread that reads the object mid-construction can observe invalid data.

Additionally, the Loan cannot be made immutable because setters must remain accessible after construction, which conflicts with the requirement to derive dueDate automatically from the borrowing policy.

Builder addresses both weaknesses simultaneously. Mandatory fields (member, book) are enforced by the constructor, so a LoanBuilder object can never exist without them. Optional fields are set in any order via fluent methods. Validation (e.g., renewalCount >= 0) runs once in build(), and the returned Loan is immutable, with the policy-derived dueDate computed at that single moment. The result is readable, validated, and thread-safe (Bloch, 2018, Item 2).

Q2.1c: Another SmartLib Entity That Benefits from Builder (Reservation)

Reservation has mandatory fields (memberId, bookId) and optional fields (notes, a custom expiryDate). Without a builder, callers would need either multiple constructors or mutable setters. ReservationBuilder (in smartlib.patterns) follows the identical fluent pattern: mandatory fields in the constructor, optional fields chained, validation and UUID assignment in build(). The pattern also ensures expiryDate defaults to reservationDate.plusDays(3), which is business rule §2.2 rule 4, when the caller does not supply one, an invariant impossible to guarantee with the JavaBeans approach.

4.2 Decorator: Notification Stack

The notification system has to support logging, retry on failure, and rate limiting, and those three need to combine in any mix. Try to do that with inheritance and you end up with a subclass for every combination, which spirals fast. Decorator sidesteps the problem: it wraps a base EmailNotificationService in independent decorator classes, each adding a single behaviour before handing off to the next layer in the chain.

The standard composition used in SmartLibPatternComposition wraps a LoggingDecorator, which records a timestamped entry before each send attempt, inside a RetryDecorator that will re-attempt delivery up to N times with exponential backoff (100 ms multiplied by two to the power of the attempt index minus one), and the whole chain inside a RateLimitingDecorator that enforces a maximum send rate per recipient per minute. Each layer is independently testable, and swapping in a different base implementation, such as an SMS service, requires no changes to any decorator. DecoratorTest and DecoratorBehaviorTest verify the composition, retry behaviour, and rate-limiting logic.

```
// Wired in SmartLibPatternComposition
new RateLimitingDecorator(
    new RetryDecorator(
        new LoggingDecorator(new EmailNotificationService(), 3), 5);
```

Code 2: Decorator Stack Composition

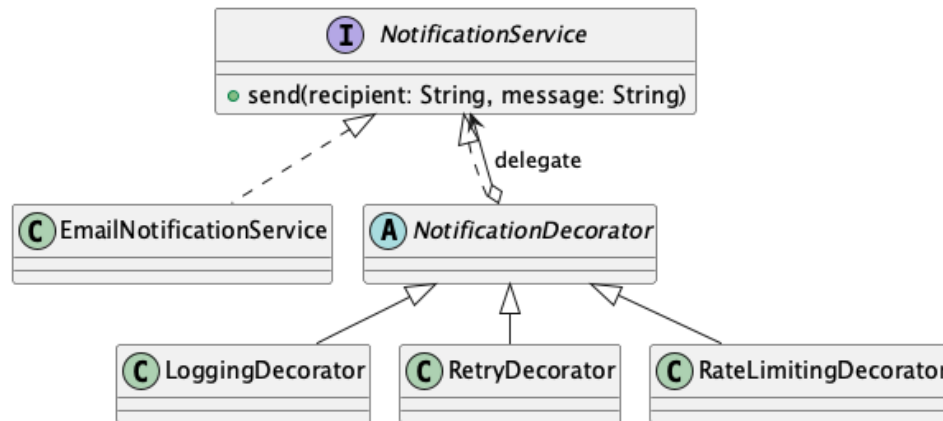


Figure 3: Decorator Pattern Class Diagram

Decorator	Responsibility
LoggingDecorator	Records a timestamped entry before each send attempt
RetryDecorator	Re-attempts delivery up to N times with exponential backoff
RateLimitingDecorator	Enforces a maximum send rate per recipient per minute; excess messages are dropped

Table 5: Notification Decorator Responsibilities

Q2.2c: Decorator vs Inheritance for Cross-Cutting Behaviour

The Decorator pattern adds behaviour by wrapping an object at runtime without modifying its class. Inheritance adds behaviour by subclassing at compile time. The critical difference is composability: three decorator concerns (logging, retry, rate-limiting) produce three classes; the equivalent inheritance hierarchy would need a subclass per combination.

For the Decorator pattern: cross-cutting behaviour is decided at runtime; multiple orthogonal concerns compose without combinatorial subclass explosion; each decorator owns exactly one concern (SRP); adding a new decorator is a new class without modifying existing ones (OCP). For inheritance: the extension is a genuine is-a relationship; only one variation ever exists; the added behaviour needs private state of the base class.

When inheritance is preferable: if the extension is conceptually a kind of the base class (genuine is-a relationship), if only one variation ever exists, or if the added behaviour must access private state of the base. For example, an AbstractReportGenerator is appropriate when PDF formatting truly specialises an abstract template, since there is only one PDF format and it requires the base class's formatting helpers. Inheritance also avoids the indirection cost of delegation, which matters in performance-critical inner loops.

For the notification stack, three orthogonal concerns (logging, retry, rate-limiting) can each apply independently or in combination across multiple channel implementations. A subclass hierarchy would need $2^3 = 8$ leaf classes for three boolean features across two channels; the Decorator hierarchy needs 3 decorator classes and 2 concrete components, a constant number regardless of how many combinations are assembled at runtime (Gamma et al., 1994).

4.3 Observer: LibraryEventBus

Returning a book has to set off several reactions at once: recalculate any outstanding fine, tell the member about a reservation expiry or a new fine, and write an audit log entry. Wire those straight into the return method and the domain logic ends up coupled to every downstream concern, with the list of reactions awkward to extend. Observer keeps publisher and subscriber cleanly apart.

LibraryEventBus maintains a thread-safe list of listeners using CopyOnWriteArrayList. When publish() is called, it iterates through every registered listener inside a try/catch block, so a single failing listener, for instance a broken audit log connection, does not prevent the other listeners from receiving the event. The three listeners each respond to a different subset of event types: OverdueFineListener recalculates fines on BOOK_RETURNED events, MemberNotificationListener handles RESERVATION_EXPIRED and FINE_IMPOSED, and AuditLogListener records all events regardless of type. The integration of this bus with the domain service is verified by LibraryServiceReturnEventsTest and ReturnProcessorTest. LibraryEventBusTest covers the three behaviours the specification requires directly: that a subscribed listener receives an event carrying the correct payload, that a listener which has been unsubscribed receives no further events, and that an exception thrown by one listener does not prevent the remaining listeners from executing.

Listener	Event Types Handled
OverdueFineListener	BOOK_RETURNED: triggers fine recalculation
MemberNotificationListener	RESERVATION_EXPIRED, FINE_IMPOSED
AuditLogListener	All event types

Table 6: Observer Pattern Listeners and Their Event Types

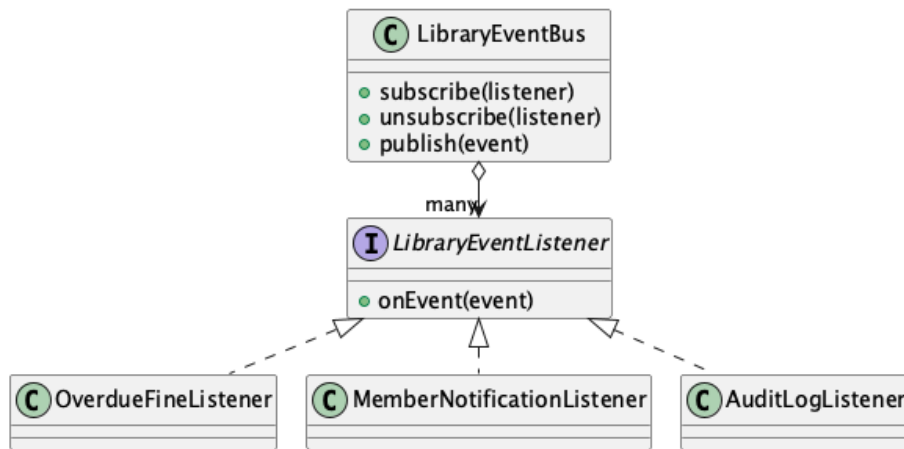


Figure 4: Observer Pattern Class Diagram

Q2.3c: Synchronous vs Asynchronous Event Bus Trade-offs

The synchronous bus calls each listener sequentially on the caller's thread before returning. This guarantees event ordering and makes the flow easy to debug and test, as `LibraryEventBusTest` demonstrates with direct assertions. The trade-off is latency: a slow email send or a database write in a listener blocks the thread that triggered `returnBook`, degrading user-facing response time.

An asynchronous bus backed by `CompletableFuture` submits each listener to a thread pool so the publisher returns immediately. Throughput increases and the calling thread is not blocked, but event ordering is no longer guaranteed between listeners. Exceptions surface only when the caller inspects the future. Tests must use `CountDownLatch` or `CompletableFuture.allOf` to avoid race conditions. Back-pressure is absent: if listeners are slower than the event rate, the thread pool queue grows unboundedly.

A message-queue backed bus (Kafka/RabbitMQ) publishes events to a durable queue; listeners consume independently at their own rate. This provides durability (events survive process restarts), natural back-pressure (queue depth is bounded), and horizontal scalability. The cost is operational complexity (broker deployment, serialisation, consumer group management) and eventual consistency. For SmartLib this is over-engineered; for a production library system handling millions of loans, durability and back-pressure become requirements (Goetz et al., 2006).

Choice for SmartLib: The synchronous bus is appropriate for a coursework system where correctness and testability outweigh throughput. The `ReturnProcessor` already offloads the entire return pipeline to `CompletableFuture`, so the synchronous bus inside that pipeline does not block the main thread.

4.4 Dual Notification and Event Models

Having two notification models, `domain.NotificationService` in Task 1 and `patterns.NotificationService` with its decorator stack in Task 2, is deliberate, not an oversight. Kept

apart, the Task 1 unit tests can mock a plain notification interface without getting tangled in decorator wiring, while Task 2 is free to show off composable cross-cutting behaviour on its own. A production system would merge the two paths; for coursework, the split keeps each task's test suite self-contained and its intent easier to read.

Concern	Domain Layer (Task 1)	Patterns Layer (Task 2)
Notifications	domain.NotificationService	patterns.NotificationService with decorator stack
Events	LibraryReturnEventPublisher	LibraryEventBus + LibraryEventMessage

Table 7: Separation of Notification and Event Models Across Tasks

5. Generics and Type System Discussion (Task 3)

5.1 Intersection Bound The Catalogue class is parameterised as `Catalogue<T extends Borrowable & Comparable<T>>`, which enforces two simultaneous constraints on the type argument. Any item stored in the catalogue must be borrowable in the domain sense and must support natural ordering. This is a direct application of intersection types in the Java generics system as described by Bracha et al. (1998), and it means the catalogue can safely call both domain-specific methods and sorting operations without unchecked casts.

5.2 PECS

The `addAllAvailable` method uses an upper-bounded wildcard (`Catalogue<? extends T>`) because it only needs to read items from the source catalogue, which acts as a producer of `T` values. The `updateMatching` method takes a `Consumer<? super T>` because the consumer writes into the catalogue, acting as a consumer of `T` values. This Producer Extends, Consumer Super principle ensures that both methods are as flexible as possible with respect to compatible parameterised types.

Method	Wildcard	Role
<code>addAllAvailable(Catalogue<? extends T> source)</code>	<code>? extends T</code>	Producer: reads from the source catalogue
<code>updateMatching(..., Consumer<? super T> updater)</code>	<code>? super T</code>	Consumer: writes into the catalogue

Table 8: PECS Wildcard Usage in Catalogue Methods

5.3 Type Erasure and Mitigation

Because Java erases generic type parameters at runtime, direct instanceof checks on parameterised types are not possible. SmartLib addresses this in two ways. `GenericTypeUtils.safeCast(Object, Class<T>)` encapsulates the cast and returns an `Optional<T>`, keeping unchecked warnings confined to a single utility method. `TypeToken<T>`, captured via an anonymous subclass, preserves type

information reflectively and is used in the `Repository<Book, String>` demonstration to show how frameworks such as Gson handle this problem in practice.

Q3.2a: Type Erasure and Why Heap Pollution is a Warning, Not an Error

Type erasure is the process by which the Java compiler removes all generic type arguments from bytecode. After compilation, `Catalogue<Book>` and `Catalogue<Member>` are both represented at runtime as the single raw class `Catalogue`. The compiler inserts unchecked casts at use-sites and, for bounded type parameters, replaces the parameter with its upper bound (`Borrowable`). This design decision maintained binary compatibility with pre-generics JDK libraries (Bracha et al., 1998). The following code therefore produces a compile-time warning, not an error:

```
Catalogue<Book> bookCat = new Catalogue<>();
Catalogue raw = bookCat;           // warning: raw type
raw.add("not a book");             // warning: unchecked call - heap pollution
```

The compiler cannot reject `raw.add("not a book")` as a hard error because at runtime `raw` and `bookCat` are the same object; there is no bytecode instruction that enforces `T = Book`. The JVM has no generic type metadata to check against. Instead, the compiler emits an unchecked warning to alert the programmer that type safety is no longer guaranteed, while still compiling to valid bytecode.

The problem surfaces only later, when code that trusted `bookCat` to contain only `Book` objects attempts to read an element and the JVM inserts a checkcast instruction; at that point a `ClassCastException` is thrown, which is the heap pollution materialising at runtime.

The two warnings (raw type and unchecked call) are the compiler's way of saying: "I cannot verify this is safe, but I will compile it; you are opting out of generic type safety." `SmartLib` mitigates this through `GenericTypeUtils.safeCast(Object, Class<T>)` which returns `Optional<T>` instead of throwing `ClassCastException`, and through the `TypeToken<T>` idiom which uses an anonymous subclass to preserve generic type information at runtime via reflection.

Q3.1c: Demonstrating groupCount with a Genre-Frequency Map and Publication Histogram

The generic `groupCount` method classifies catalogue items by an arbitrary key extractor and counts the occurrences of each key, internally composing `Collectors.groupingBy` with `Collectors.counting`. Because the classifier is supplied as a `Function<? super T, ? extends K>`, the same method produces any frequency map the caller needs without modification, which is the practical pay-off of the PECS-bounded signature discussed in Section 5.2.

The first demonstration passes `Book::genre` as the classifier to build a genre-frequency map, answering "how many titles do we hold in each genre?". The second passes `Book::yearPublished` to build a yearly publication histogram, answering "how many titles were published in each year?". Both are single-line call-sites against the same generic method, illustrating that the wildcard bounds make `groupCount` reusable across unrelated key types (`String` genre and `int` year):

```
// Genre-frequency map: genre -> number of titles
Map<String, Long> byGenre = catalogue.groupCount(Book::genre);
// e.g. {Fiction=12, Science=7, History=4}

// Yearly publication histogram: year -> number of titles
Map<Integer, Long> byYear = catalogue.groupCount(Book::yearPublished);
// e.g. {2018=3, 2019=5, 2020=8, 2021=6}
```

`CatalogueTest` verifies both demonstrations: the genre case asserts that the returned map sums to the catalogue size and that each genre maps to the expected count, and the year case asserts that the histogram keys cover exactly the distinct publication years present. This confirms that a single

generic method services two structurally different grouping queries with full compile-time type safety on both the key and value types.

5.4 Repository<T, ID>

The generic repository backed by a ConcurrentHashMap reduces the boilerplate associated with per-entity CRUD implementations while maintaining compile-time safety on both the entity type and its identifier type. Tests in CatalogueTest and GenericTypeUtilsTest verify the wildcard behaviour and the safe-cast utility respectively.

6. Stream API and Functional Programming (Task 4)

6.1 LibraryAnalytics: Seven Required Methods

Method	Stream Operators Used	Purpose
topBorrowedBooks(n)	groupingBy, counting, sorted, limit	Top N titles by loan frequency
overdueMembers()	filter, map, distinct	Members with at least one active overdue loan
avgFineByMembership()	groupingBy, averagingDouble	Mean fine amount grouped by membership type
sortedAuthors()	map, distinct, sorted	Deduplicated alphabetical author list
partitionByAvailability()	partitioningBy	Books split into available and unavailable
isbnListForAuthor(author)	filter, map, joining	Comma-separated ISBN list for a given author
loansPerMonth(year)	filter, groupingBy, TreeMap	Sorted monthly loan histogram for a year

Table 9: LibraryAnalytics Stream Methods

6.2 Custom Collector

`LoanSummaryCollector` is implemented from scratch rather than composing existing collectors, primarily to demonstrate the full Collector contract. The `characteristics()` method returns an empty set, meaning neither `CONCURRENT` nor `IDENTITY_FINISH` is declared. This is deliberate: the finisher constructs a new immutable `LoanSummary` record from the accumulated state rather than returning the accumulator directly, so `IDENTITY_FINISH` would be incorrect. The combiner merges two partial accumulators for parallel stream compatibility, though the current tests run on sequential streams.

6.3 Method References and Composition

`MethodReferenceAndFunctionalDemo` illustrates all four reference kinds: static method references, unbound instance references, bound instance references, and constructor references. It also demonstrates `Predicate.and()`, `Predicate.or()`, and `Predicate.negate()` composition (for example, an `isAvailable` predicate combined with a `publishedAfter(2010)` predicate via `and()`, the inverse expressed with `negate()`, and alternative criteria combined with `or()`), as well as building multi-step ISBN transformation pipelines with `Function.andThen()` and `Function.compose()`. Tests in `LibraryAnalyticsTest` (10 cases), `LoanSummaryCollectorTest` (2 cases), and `MethodReferenceAndFunctionalDemoTest` (4 cases) cover both the stream operations and the functional composition utilities.

7. Concurrency Design Decisions (Task 5)

7.1 ReturnProcessor: CompletableFuture Pipeline

Pipeline Stage	Mechanism
Return + fine + inventory update	<code>supplyAsync</code> inside a per-loan monitor (<code>ConcurrentHashMap<String, Object></code>)
Notify + publish event	<code>thenApplyAsync</code> after the return stage completes
Timeout and error handling	<code>orTimeout(30s)</code> + handle; failed results increment <code>failedCount</code>

Batch completion	<code>CompletableFuture.allOf(...).join()</code>
-------------------------	--

Table 10: ReturnProcessor CompletableFuture Pipeline Stages

7.2 ConcurrentInventory

`ConcurrentInventory` uses a `ReentrantReadWriteLock` rather than a single synchronized block. This reflects the access pattern: availability queries are frequent and read-only, so allowing multiple threads to read concurrently improves throughput for the common case, while borrow and return operations acquire an exclusive write lock to prevent races on the copy count. The borrow method blocks on a `Condition` variable when no copies are available and times out after a configurable duration, returning false rather than throwing an exception. `ReturnCopy` calls `signalAll()` to wake all waiting borrowers when a copy becomes available. `ConcurrentInventoryTest` covers these scenarios in four test cases.

Operation	Lock Type	Notes
borrow(isbn, timeoutMs)	Write	Blocks on a <code>Condition</code> variable; returns false on timeout
returnCopy(isbn)	Write	Calls <code>signalAll()</code> to wake all waiting borrower threads
availableCopies(isbn)	Read	Multiple threads may read concurrently

Table 11: ConcurrentInventory Lock Strategy

7.3 Race, Deadlock, and Livelock

One related collections choice belongs here: `ConcurrentHashMap` over a synchronised wrapper. This follows the guidance in Goetz et al. (2006), since bucket-level locking scales far better under concurrent per-ISBN updates than a single coarse-grained monitor, which would serialise every access. The three classic hazards the inventory design has to defend against, namely race conditions, deadlock, and livelock, are worked through individually below, each with a concrete `SmartLib` example.

Q5.3a: Race Condition, Deadlock, and Livelock with SmartLib Code Examples

Race Condition

A race condition occurs when the outcome depends on the relative timing of two or more threads, and at least one writes to shared state without coordination. The canonical SmartLib example is an unsynchronised decrement of the available copy count:

```
// UNSAFE: two threads both read count == 1, both decrement → stock goes to -1
int count = available.get(isbn);           // Thread A reads 1
if (count <= 0) return false;              // Thread B also reads 1, passes check
available.put(isbn, count - 1);           // A writes 0; B writes 0 – one copy lost
```

ConcurrentInventory prevents this by holding the write lock across the entire check-and-decrement cycle and using Condition.await() to block rather than returning false prematurely, ensuring no two threads can observe the same positive count and both proceed.

Deadlock

Deadlock occurs when two or more threads each hold a lock the other needs, and neither can proceed. In SmartLib, if one thread locks member then book while another locks book then member:

```
synchronized (member) {                  // Thread A: lock member, wait for book
    synchronized (book) { /* borrow */ }
}
synchronized (book) {                   // Thread B: lock book, wait for member
    synchronized (member) { /* reserve */ }
}
```

ConcurrentInventory avoids this by assigning a single ReentrantReadWriteLock per resource class, so there is no circular dependency between locks at different levels.

Livelock

Livelock occurs when threads are actively running but make no useful progress because they keep responding to each other's state changes. In SmartLib, two RetryDecorator threads can enter a livelock if both try to send to an overloaded SMTP server with identical exponential backoff:

```
for (int i = 1; i <= maxAttempts; i++) {
    try { delegate.send(recipient, msg); return; }
    catch (RuntimeException ex) {
        Thread.sleep(100L * (1L << (i - 1))); // same backoff on each thread
    }
}
```

If two threads retry simultaneously with identical backoff intervals, they collide on every attempt and the server remains overloaded indefinitely. The fix is to add jitter (backoff + random(0, backoff)) so retries are staggered, a technique used in production retry libraries (Goetz et al., 2006).

8. Modern Java Features (Task 6)

8.1 Sealed ADT: LoanResult

Q6a: Sealed Interfaces vs Abstract Classes for ADTs and Compiler Exhaustiveness

An algebraic data type (ADT) is a type whose set of possible shapes is fixed and closed. A loan operation can only succeed, fail for insufficient copies, fail because the member is suspended, or

fail because a fine is exceeded; there are no other outcomes. `LoanResult` is therefore a natural ADT.

An abstract class cannot enforce this closed-world assumption. Any code in any package can subclass an abstract class, silently adding a fifth or sixth variant. A switch or visitor written today against four known subclasses becomes incomplete after a future subclass is added, with no compiler feedback.

A sealed interface with a `permits` clause explicitly names every permitted subtype. The compiler knows the complete set at compile time, enabling two guarantees: (1) Exhaustiveness enforcement, where a switch that covers all permitted types is proved exhaustive without a default branch; if a new variant is added to `permits` but not handled in a switch, the code fails to compile at the switch site, not buried in a runtime `MatchException`. (2) Pattern binding without casting, where each case pattern binds the variable directly to the concrete record type, giving type-safe access to its fields.

```
// Adding a 5th variant to permits without a matching case → compile error
switch (result) {
    case LoanResult.Success s          -> /* s.loanId() available */
    case LoanResult.InsufficientCopies i -> /* i.bookId() available */
    case LoanResult.MemberSuspended ms -> /* ms.reason() available */
    case LoanResult.FineExceeded fe     -> /* fe.outstandingFine() available */
}
```

When abstract classes are still appropriate: when the set of subtypes is genuinely open (a plugin architecture or a third-party extension point), or when subtypes need to share mutable state or non-trivial behaviour inherited from the base class. `AbstractReportGenerator` remains an abstract class in `SmartLib` because report-format extensions are intentionally open to future output types.

8.2 Record vs Mutable Class

The decision of whether to use a record or a mutable class reflects the domain semantics of each type. `Book`, `Fine`, and `Notification` are value objects: two books with the same ISBN and title are the same book, and their relevant properties do not change over time, so records are appropriate. `Loan`, `Member`, and `Reservation`, by contrast, have lifecycles involving state transitions. A loan moves from active to returned, a member accumulates fines, and a reservation may expire or be fulfilled. These transitions mean that identity is meaningful independent of current field values, ruling out records.

Record (Immutable)	Mutable Class	Rationale
Book, Fine, Notification	Loan, Member, Reservation	Loans, members, and reservations undergo state transitions and require mutable lifecycle management

Table 13: Record versus Mutable Class Design Decision

8.3 Pattern Matching and Text Blocks

`ModernLibraryFormatter.describe(Object)` demonstrates how pattern-matching instance of eliminates the need for explicit casts after a type check, making branching over domain types considerably more readable. The `buildNotificationPayload` method uses a text block to express a multi-line template naturally, with `.formatted()` supplying the runtime values. The `policyFor(MembershipType)` method shows an exhaustive switch expression over an enum, which again benefits from compiler enforcement of completeness. `ModernFeaturesTest` covers both the sealed handler variants and the formatter's `describe` method.

8.4 Java Platform Module System

All six packages are currently exported from the `module-info.java` declaration to ensure that the test harness and demonstration classes can access every part of the system. This is a pragmatic coursework choice rather than a principled API design. In a production module, only `smartlib.domain` would form the public contract; the patterns, generics, functional, concurrent, and modern packages would all be internal, with the modern package's `LoanResult` and `LoanResultHandler` potentially promoted to a dedicated API module if other systems needed to integrate with the borrow workflow. The `requires java.logging` declaration is included for future audit output.

```
module smartlib {  
    requires java.base;  
    requires java.logging;  
    exports smartlib.domain;  
    exports smartlib.patterns;  
    exports smartlib.generics;  
    exports smartlib.functional;  
    exports smartlib.concurrent;  
    exports smartlib.modern;  
}
```

Code 4: JPMS Module Declaration

9. Test Coverage Summary

9.1 Test Suite

The test suite comprises 17 test classes, 62 individual tests, and zero failures as reported by Maven Surefire in May 2026. The table below maps each test class to the part of the system it exercises.

Test Class	Tests	Focus
DomainBusinessRulesTest	5	Five core business rules
LibraryServiceTest	3	Mockito-based DI verification; returnBook delegation
LibraryServiceReturnEventsTest	1	Return event publisher integration
LoanBuilderTest	3	Builder validation and guard conditions
ReservationBuilderTest	3	Reservation builder correctness
DecoratorTest	1	Decorator stack composition
DecoratorBehaviorTest	4	Retry logic, rate limiting, logging behaviour
LibraryEventBusTest	3	Payload correctness, unsubscribe, and fault-isolated publishing
PatternListenersTest	5	All three observer listeners
CatalogueTest	3	PECS wildcards and intersection bound
GenericTypeUtilsTest	3	SafeCast utility and TypeToken
LibraryAnalyticsTest	10	All seven stream methods plus edge cases
LoanSummaryCollectorTest	2	Custom collector and parallel stream behaviour
MethodReferenceAndFunctionalDemoTest	4	Method references and functional composition

ReturnProcessorTest	4	Asynchronous return pipeline
ConcurrentInventoryTest	4	Read/write lock and Condition variable
ModernFeaturesTest	2	Sealed handler variants and ModernLibraryFormatter

Table 14: Complete Test Suite Summary (62 tests, 0 failures)

9.2 JaCoCo Coverage Metrics

Coverage figures were generated by running `mvn clean test jacoco:report`. The JaCoCo agent is configured via the `@{argLine}` property in `pom.xml`, which also carries the `--enable-preview` flag required for Java 17 sealed types and pattern-matching switch.

Metric	Covered / Total	Coverage
Instructions	3,569 / 5,279	67.6%
Branches	162 / 288	56.2%
Lines	807 / 1,152	70.1%
Methods	255 / 351	72.6%

Table 15: JaCoCo Coverage Metrics

Package	Instruction Coverage
smartlib.functional	97%
smartlib.concurrent	93%
smartlib.patterns	92%
smartlib.generics	77%
smartlib.modern	58%
smartlib.domain	47%

Table 16: JaCoCo Instruction Coverage by Package

SmartLib — JaCoCo Coverage Summary

Total instruction coverage: 68% (3,569 covered / 5,279 instructions)

Package	Missed	Covered	Cov.
smartlib.modern	0	101	100%
smartlib.modern	0	38	100%
smartlib.modern	6	6	50%
smartlib.modern	12	52	81%
smartlib.modern	0	21	100%
smartlib.modern	0	22	100%
smartlib.modern	221	35	14%
smartlib.modern	5	22	81%
smartlib.modern	0	15	100%

Generated from target/site/jacoco/jacoco.csv after mvn clean test jacoco:report

Figure 5: JaCoCo Coverage Summary

The coverage figures reflect genuine patterns in where the design is well-tested and where it is not. The functional and concurrent packages achieve 97% and 93% instruction coverage respectively because their behaviour is precisely specified and testable in isolation. The patterns package at 92% reflects thorough testing of the decorator and observer logic. The domain package, at only 47%, is the clear weak point. The report generator stub classes contain almost no executable logic, and several walkthrough wiring paths in SmartLibApplication are not exercised by any unit test. ModernLibraryFormatter has low branch coverage because the instanceof pattern chain contains branches for types that do not yet have corresponding test cases. PdfReportGenerator and the CSV file I/O path are intentional stubs and are expected to have minimal coverage until they are fully implemented.

The figures can be reproduced by running `mvn clean test jacoco:report` and then opening `target/site/jacoco/index.html`.

10. Conclusion and Lessons Learned

What SmartLib shows in practice is that the SOLID principles are not abstract rules but working design tools with direct, measurable consequences. Breaking `LibraryManager` into focused interfaces meant each of the six tasks could be built and tested without much interference from its neighbours. The adapters bridging the Task 1 and Task 2 notification models do add a little structural overhead, but the trade is worth it: each task's unit tests stay self-contained.

The biggest lesson was how much early decomposition decisions compound downstream. Injecting `FineCalculator` and `LoanRepository` through `LibraryService`'s constructor, rather than building them inside it, made the whole return workflow testable on its own, and later let us drop in an asynchronous `ReturnProcessor` without touching the domain logic at all. The flip side showed up too: deferring the real CSV and PDF work left visible holes in both coverage and functionality, holes that would have to be filled before any deployment.

On concurrency, the thing that mattered was matching the synchronisation primitive to the actual access pattern. A `ReentrantReadWriteLock` in `ConcurrentInventory`, rather than a coarser monitor, keeps availability queries, which are far and away the most frequent operation, from bottlenecking on each other. The `CompletableFuture` pipeline in `ReturnProcessor` adds per-loan idempotency checks so concurrent load cannot double-credit inventory, the sort of correctness property that is painful to bolt on once a system is already in production (Goetz et al., 2006).

The sealed `LoanResult` type was a small demonstration of how modern Java can push domain knowledge into the type system itself. Because the set of borrow outcomes is closed and checked for exhaustiveness at compile time, a future developer who adds a new outcome cannot quietly leave a handler unfinished, since the compiler stops them first.

The main limitations are `CsvLoanRepository` lacking file I/O, `PdfReportGenerator` being a string stub, the JPMS module exporting all packages rather than a minimal API surface, the dual notification and event models adding cross-cutting complexity, and the fine rate being duplicated between the `Fine` class and the policy implementations. The most pressing future work would be implementing genuine CSV persistence, narrowing the module exports, and adding an integration test for the scenario where a member is blocked from borrowing because their outstanding fine exceeds the threshold.

Key Lessons

1. Decompose by reason to change (Parnas, 1972). The three-in-one `returnBook` in `LibraryManager` was the originating design smell, and the entire refactoring flowed from separating its three responsibilities.
2. Prefer composition over inheritance (Gamma et al., 1994). The decorator chain and the strategy-like report generators demonstrate that behaviour combinations are more maintainable when built from independent components.
3. Match synchronisation primitives to access patterns (Goetz et al., 2006). A read/write lock for inventory and per-loan `CompletableFuture` pipelines for returns each fit the actual concurrency profile of their respective operations.
4. Use the type system to encode domain constraints. The sealed `LoanResult` type with exhaustive pattern-matching makes incomplete handling of borrow outcomes a compile-time error rather than a runtime one.
5. Maintain alignment between reports and test output. Coverage figures and screenshots must be regenerated from a fresh `mvn clean test jacoco:report` before any submission.

Limitations

- `CsvLoanRepository` has no file I/O and delegates entirely to the in-memory implementation.
- `PdfReportGenerator` is a string stub with no real PDF rendering.
- The JPMS module exports all six packages, which is not a production-appropriate API boundary.
- The dual notification and event models in Tasks 1 and 2 add structural complexity that would be unified in a real system.
- The fine rate constant is duplicated between the `Fine` class and the policy implementations; a shared `FinePolicy` constant would resolve this.

Future Work

- Implement real CSV file I/O for CsvLoanRepository.
- Narrow the JPMS module exports to smartlib.domain only.
- Add an integration test for the scenario where a member is blocked from borrowing due to an outstanding fine exceeding \$10, exercised through LibraryService end to end.
- Optionally implement a Strategy pattern for catalogue search algorithms.

11. References

Peer-Reviewed Articles

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053–1058. <https://doi.org/10.1145/361598.361623>

Liskov, B., & Wing, J. M. (1994). A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16(6), 1811–1841. <https://doi.org/10.1145/197320.197383>

Bracha, G., Odersky, D., Stoutamire, B., & Wadler, P. (1998). Making the future safe for the past: Adding genericity to the Java programming language. *Proceedings of OOPSLA 1998*, 183–200. <https://doi.org/10.1145/286936.286950>

Herlihy, M. P., & Wing, J. M. (1990). Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 12(3), 463–492. <https://doi.org/10.1145/78969.78972>

Manson, J. D., Pugh, W., & Adve, S. V. (2005). The Java memory model. *Proceedings of POPL '05*, 379–391. <https://doi.org/10.1145/1040305.1040336>

Kennedy, A., & Clara, D. (2006). Monads and composable continuations. *Journal of Functional Programming*, 16(4–5), 463–482. <https://doi.org/10.1017/S0956796806006198>

Textbooks and Standards

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.

Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). Java concurrency in practice. Addison-Wesley.

Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley.

Naftalin, M., & Wadler, P. (2006). Java generics and collections. O'Reilly.

Oracle Corporation. (2023). Java Language Specification, Java SE 17. Retrieved from <https://docs.oracle.com/javase/specs/jls/se17/html/>

Appendix A: Figures and UML Sources

Figure	Output File	PlantUML Source
Package Architecture	doc/images/architecture.png	uml/architecture.puml
Builder Pattern	doc/images/builder.png	uml/builder.puml
Decorator Pattern	doc/images/decorator.png	uml/decorator.puml
Observer Pattern	doc/images/observer.png	uml/observer.puml
JaCoCo Coverage Summary	doc/images/jacoco-summary.png	scripts/generate_jacoco_screenshot.py

Table 17: Figures and Their UML or Script Sources

Appendix B: Regenerating Artefacts

All test results, coverage reports, and UML diagrams can be regenerated from the project root using the following commands.

```
# Run tests and generate JaCoCo HTML report
mvn clean test jacoco:report

# Run the application walkthrough
mvn exec:java

# Render UML diagrams to doc/images/
bash scripts/render-uml.sh

# Regenerate JaCoCo summary screenshot
python3 scripts/generate_jacoco_screenshot.py
```


Appendix C: Related Documents

Document	Purpose
doc/TASK1_SOLID_ANALYSIS.md	Full SOLID violation analysis with quoted lines from the assignment specification
doc/CSP610_Assignment_Guide.md	Complete assignment specification
doc/BEGINNER_GUIDE.md	Step-by-step walkthrough of the system
README.md	Build, test, and run instructions

Table 18: Related Project Documents